SUZOHAPP

OrchardJ Software Development Kit Manual

Version 2.0.0

Document History

Version	Changes
2.0.0	× See chapter **SDK 2.0.0** for a description of all the news in version 2.0.0 of the SDK × OrchardJ 3.0.0 introduces icons for the services, see chapter **Icons for services** for more details
1.4.1	× Added move notes to transport box
1.4.0	× Support for all variants of RCS 400 × Simplified dispense handling. Deprecated the concept of “dispense destination” for coins. × New door handling that supports more machine types. × Support for more variants of I/O boards (e.g. RCS 400 and CDS9)
1.3.9	× Added getExpectedAmount() on IOJUser with option to get remaining or originally set amount(s)
1.3.8	× Added setExpectedAmount() on IOJUser to allow setting multiple expected amounts by Currency
1.3.7	× Added getExpectedAmount() and setExpectedAmount() on IOJUser to allow to set expected amounts for the user during the standard OrchardJ deposit flow.
1.3.6	× Added IOJKeyListener2 for support for custom screens
1.3.5	× UNLIMITED IOJDispenseDestination added
1.3.2	× IOJTransaction2 added
1.3.1	× Transaction with additionalMoney always created in doDeposit. × CAM Error 138 handled correctly
1.3.0	× Added possibility to hide OrchardJ windows
1.2.0	× Added support for work unit × Added possibility of having a margin to the left of barcodes printed with OJReceipt.addBarcode() × Added possibility to cancel IOJDepositManager.beginTransaction() and IOJDispenseManager.doDispense() when they are waiting for till to be inserted/removed. × Support for rear access (IOJOperatorLoginHandler2 and automatic printer selection)
1.1.1	× Added new transaction types (IOJTransaction.OJTransactionType) × Since release 1.10.0 of OrchardJ RCS Software the restriction that JRE 1.6.0_u11 or newer was not allowed

USER LICENSE AGREEMENT FOR SUZOHAPP SOFTWARE IMPORTANT — READ CAREFULLY:

The OrchardJ Software Development Kit is provided under SUZOHAPP's general terms of delivery. The copyright and all other intellectual property rights in the Current Release, and all corrected versions thereof and all other Releases, and the Program Documentation are and shall remain the property of SUZOHAPP. The Current Release shall not, without the prior written agreement with SUZOHAPP, be altered or modified in any way whatever, including screens and logotypes, nor be combined with other programs to form a combined work. The Current Release may not be transferred or leased to any third party prior written agreement with SUZOHAPP.

	has been removed (you can now run with any JRE 1.6)
1.1.0	× IOJCashDevice.getDepositHandler() can be called from monitoring flows and from isVisible() and isEnabled() in the service flow (but no methods in the deposit handler can be called, this can only be used to check the deposit type). × Added setLocale() and resetLocale() to IOJSystemManager. × Added the interface IOJCashBoxUpdater that allows updates of cash box content (empty and refill cassettes). × Deprecated printCustomReceipt() in IOJPrinter and replaced it with printReceipt(). Added classes OJAbstractReceipt and OJReceipt. × Added checks to make sure that no flow, including the monitoring flow, is started until all devices are running. × “Extra Data” added to drop safe transactions as well. × Added possibility to present decimals in inputNumber (IOJUIManager). × Added method getStandardReceiptProperties() to IOJPrinter.
1.0.0	First official release
0.2.1	Minor updates after review
0.2.0	First beta version

USER LICENSE AGREEMENT FOR SUZOHAPP SOFTWARE IMPORTANT — READ CAREFULLY:

The OrchardJ Software Development Kit is provided under SUZOHAPP's general terms of delivery. The copyright and all other intellectual property rights in the Current Release, and all corrected versions thereof and all other Releases, and the Program Documentation are and shall remain the property of SUZOHAPP. The Current Release shall not, without the prior written agreement with SUZOHAPP, be altered or modified in any way whatever, including screens and logotypes, nor be combined with other programs to form a combined work. The Current Release may not be transferred or leased to any third party prior written agreement with SUZOHAPP.

Table of contents

Introduction 6
About the SDK 6
About the documentation 6
Compatibility notes 6
Getting started Tutorial 7
Getting started with JDK 7
Setting up the workspace 7
Implementing the plug-in 7
Compiling the plug-in 9
Installing the plug-in 9
Getting started with Eclipse 9
Setting up the workspace 9
Implementing the plug-in 12
Compiling the plug-in 13
Installing the plug-in 14
Inside the SDK 15
Environment requirements 15
How to compile plug-ins 15
How to install plug-ins 15
Architecture 16
Flows 16
Boot Flow 17
Logged-in flow 17
Service flow 17
Monitor flow 18
Flow diagram 18
Managers 19
OrchardJ sessions 19
Permissions 20
The operator service menu 20
Integrating external devices 21
Troubleshooting 22
Plug-in does not seem to start 22
There is no button for plug-in service 22
OrchardJ does not seem to start 22
IOJDepositManager.beginTransaction() seems to hang 22
The SDK Examples 23
How to compile and install examples 23
Example plug-ins 24
ReceiptPrinter 24
DoorHandler 24
BagDropper 24
ExternalUserLogin 24
ExternalLoginDevice 25
DispenseValue 25
RemoteController 25
ParallelDeposit 26
CustomInfoDeposit 26
Exchange 26
TableEditor 26
Icons for services 28

SDK 2.0.0 29
New UI framework 29
Launching known flows 29
Miscellaneous 30
Permissions 30
Custom data 31
Money mix algorithm 31
Use OrchardJ trace files 32
Use OrchardJ configurations 32

Introduction

About the SDK

The OrchardJ Software Development Kit (from now on called the SDK) was first released with OrchardJ RCS version 1.8.0. The SDK consists of a lib with java interfaces that makes it possible to implement plug-ins that extend or change the functionality of OrchardJ.

To write plug-ins using the SDK you will need to be familiar with the Java programming language. However, the SDK does not use any advanced techniques so basic Java knowledge should be enough.

An SDK plug-in can register functionality that will be called by OrchardJ at different points in time, e.g. at start-up, when a user logs in or when the user selects a button on the select service page. When that happens the plug-in is called by OrchardJ and is given access to interfaces that lets it interact with the inner parts of OrchardJ.

With the SDK you can extend and change the OrchardJ software platform in many ways. This list shows just a few examples of applications for the SDK:

- Implement a new user login procedure using an external user database rather than that included in the RCS.
- Implement a new user login device, such as an RFID tag reader.
- Implement a new deposit flow for a new type of device, e.g. a cheque reader.
- Monitor the RCS and connect it to an already existing monitoring- or accounting system.
- Disable the RCS user interface and use it only as a remotely controlled coin recycler.
- Extend OrchardJ to be used as vendor independent software for a completely new type of application.

About the documentation

The SDK documentation consists of three parts:

- This manual which gives an overview of the SDK and helps you getting started, but that doesn't explain all parts of the SDK in detail.
  Location: sdk\doc\OrchardJSDKManual.pdf
- The javadoc API reference documentation explains all interfaces, classes and methods in detail.
  Location: sdk\doc\api\index.html
- There are a couple of examples that shows different ways to use the SDK.
  Location: sdk\examples\com\scancoin\sdk\examples

This manual has four main chapters. This first chapter (that you are reading) only gives a short introduction to the SDK. The next chapter is a short getting started tutorial that shows you how to write, compile and install a simple plug-in with or without Eclipse[1]. That chapter is followed by a chapter that explains the SDK in more detail, and finally the last chapter gives an explanation to the different examples.

Compatibility notes

Version 2.0.0 of OrchardJ SDK requires OrchardJ 3.0.0 or later.

[1] Eclipse is an Open-Source freeware Java Integrated Development Environment (IDE)

Getting started Tutorial

Getting started with JDK

You must have JDK version 1.6 and you must have the Path environmental variable set to point at the bin folder of the JDK installation before starting this tutorial.

Setting up the workspace

1. Start by creating a working directory, for example:
   `C:\workspaces\oj_sdk_tutorial`
2. Create three subfolders in the work dir:
   `src`
   `bin`
   `lib`
3. Copy the sdk lib (com.scancoin.sdk.jar) to the lib folder. You can find it on the RCS in the OrchardJ program folder under tools\sdk\lib (e.g. C:\OrchardJ\tools\sdk\lib)
4. Decide a package name for your plug-in, for example:
   `com.mycompany.ojsdktutorial`
5. Create a directory structure based on the package name under the src folder, e.g:
   `C:\workspaces\oj_sdk_tutorial\src\com\mycompany\ojsdktutorial`

Implementing the plug-in

6. Inside the ojsdktutorial folder you create a new text file which you call
   `HelloWorld.java`
7. Open the file in a text editor and add the package name and the necessary imports:

```
package com.mycompany.ojsdktutorial;

import java.util.HashMap;

import com.scancoin.sdk.oj.IOJFlowSession;
import com.scancoin.sdk.oj.IOJRegistrator;
import com.scancoin.sdk.oj.IOJUIManager;
import com.scancoin.sdk.oj.OJTooManyFlowsException;
import com.scancoin.sdk.plugin.IOJFlow;
import com.scancoin.sdk.plugin.IOJPlugin;
```

8. Create the main class of the plug-in. It should implement the `IOJPlugin` interface and its method `initialize()`:

```
public class HelloWorld implements IOJPlugin {
    public void initialize(IOJRegistrator registrator) {
    }
}
```

9. Now implement a private internal class called `HelloWorldFlow` which implements the interface `IOJFlow` and its method `run()`:

```
    private class HelloWorldFlow implements IOJFlow {
        public void run(IOJFlowSession session) {
        }
    }
```

10. Register the `HelloWorldFlow` as a boot flow in the initialize method of `HelloWorld`:

```
public void initialize(IOJRegistrator registrator) {
    try {
        registrator.registerBootFlow(new HelloWorldFlow());
    } catch(OJTooManyFlowsException e) {
    }
}
```

11. Implement the `run()` method in `HelloWorldFlow`. This will be called from OrchardJ at start-up and what we want to do is to show a screen saying “Hello World” with a Continue button and we want to wait for the user to press that button before proceeding to the normal login page. First we need to define a map (`HashMap`) with the key and key text we want to show, then we need to show the screen and after that we will wait for the user to press the key:

```
public void run(IOJFlowSession session) {
    HashMap<IOJUIManager.OJKey, String> key =
        new HashMap<IOJUIManager.OJKey, String>();
    key.put(IOJUIManager.OJKey.KEY_OK, "Continue");
    session.getUIManager().showTextScreen("", "Hello World",
            null, key);
    session.getUIManager().waitForKey(false, 0);
}
```

12. The entire file should look like this now:

```
package com.mycompany.ojsdktutorial;

import java.util.HashMap;

import com.scancoin.sdk.oj.IOJFlowSession;
import com.scancoin.sdk.oj.IOJRegistrator;
import com.scancoin.sdk.oj.IOJUIManager;
import com.scancoin.sdk.oj.OJTooManyFlowsException;
import com.scancoin.sdk.plugin.IOJFlow;
import com.scancoin.sdk.plugin.IOJPlugin;

public class HelloWorld implements IOJPlugin {
    public void initialize(IOJRegistrator registrator) {
        try {
            registrator.registerBootFlow(new HelloWorldFlow());
        } catch(OJTooManyFlowsException e) {
        }
    }

    private class HelloWorldFlow implements IOJFlow {
        public void run(IOJFlowSession session) {
            HashMap<IOJUIManager.OJKey, String> key =
                new HashMap<IOJUIManager.OJKey, String>();
            key.put(IOJUIManager.OJKey.KEY_OK, "Continue");
            session.getUIManager().showTextScreen("", "Hello World",
                    null, key);
            session.getUIManager().waitForKey(false, 0);
        }
    }
}
```

Compiling the plug-in

13. Open a command prompt and go to the project folder:
    `cd C:\workspaces\oj_sdk_tutorial`
14. Compile the plug-in like this:
    `javac -sourcepath src -d bin -classpath lib\com.scancoin.sdk.jar src\com\mycompany\ojsdktutorial\HelloWorld.java`
15. Now we need to go to the bin folder…
    `cd bin`
16. … and pack the plug-in into a jar file
    `jar cf HelloWorld.jar com`
17. You will find the compiled and packaged plug-in here:
    `C:\workspaces\oj_sdk_tutorial\bin\HelloWorld.jar`

Installing the plug-in

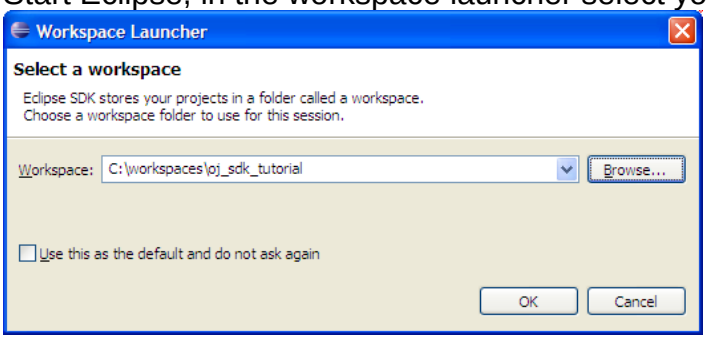
18. To install the plug-in on an RCS you first need to copy the jar file on to a memory stick (unless you have network access on the RCS).
19. Shut down the RCS software.
20. Open a file explorer on the RCS and locate the OrchardJ program folder, normally:
    `C:\OrchardJ`
21. Go to the folder data, e.g.:
    `C:\OrchardJ\data`
22. Unless it already exists you should create a new folder called:
    `plugins`
23. Copy the plug-in (HelloWorld.jar) into that folder (plugins)
24. Now locate this configuration file:
    `resources\configurable\configuration\com.scancoin.sdk.engine`
25. Open that file in a text editor (e.g. Notepad), remove all content and replace it with:
    `plugins = data/plugins/HelloWorld.jar;com.mycompany.ojsdktutorial.HelloWorld`
26. Start the RCS, you should now see the Hello World screen when it has started.

Getting started with Eclipse

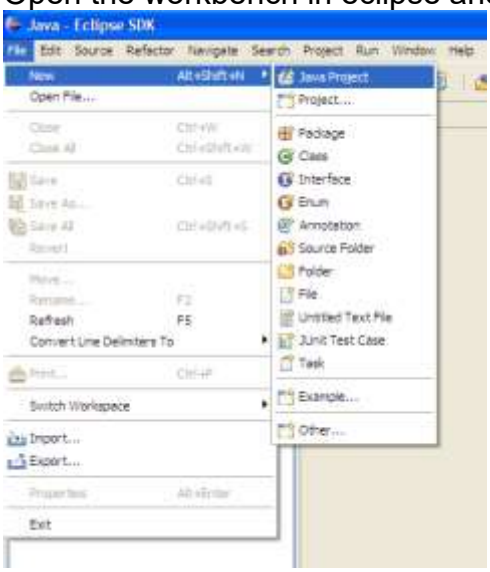
You must have Eclipse installed and setup for Java development with Java SE 6 (JDK 1.6) before starting this tutorial. This tutorial shows the steps and screens for Eclipse version 3.3.0, but it should be about the same with other versions.

Setting up the workspace

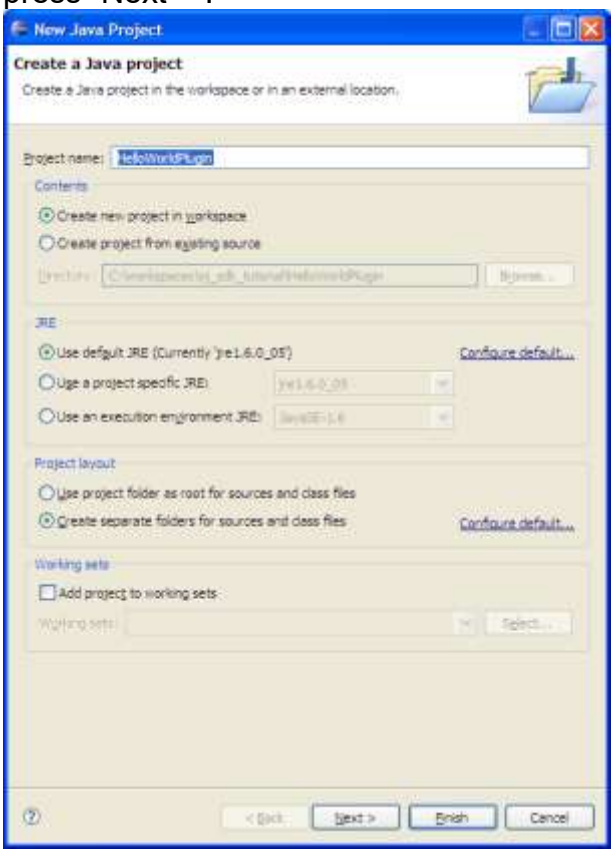
1. Start by creating a working directory, for example:
   `C:\workspaces\oj_sdk_tutorial`
2. Copy the sdk lib (com.scancoin.sdk.jar) to this folder. You can find it on the RCS in the OrchardJ program folder under tools\sdk\lib (e.g. C:\OrchardJ\tools\sdk\lib)
3. Start Eclipse, in the workspace launcher select your working directory:



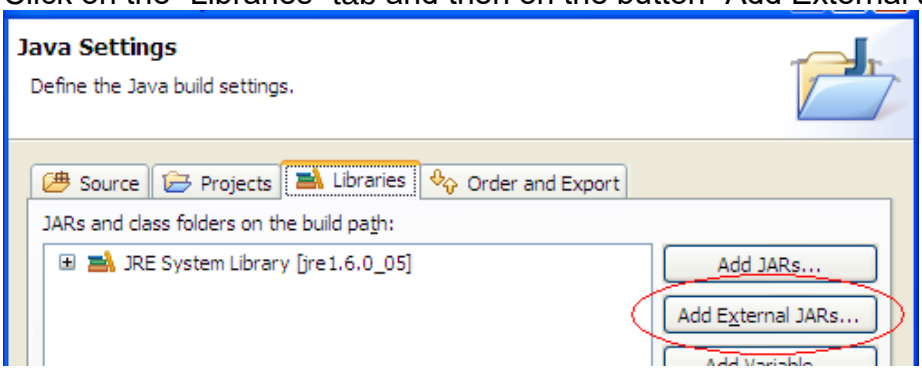
4. Open the workbench in eclipse and select “File->New->Java Project” in the menu



5. Give the project the name HelloWorldPlugin, leave all other settings as they are and press “Next >”:

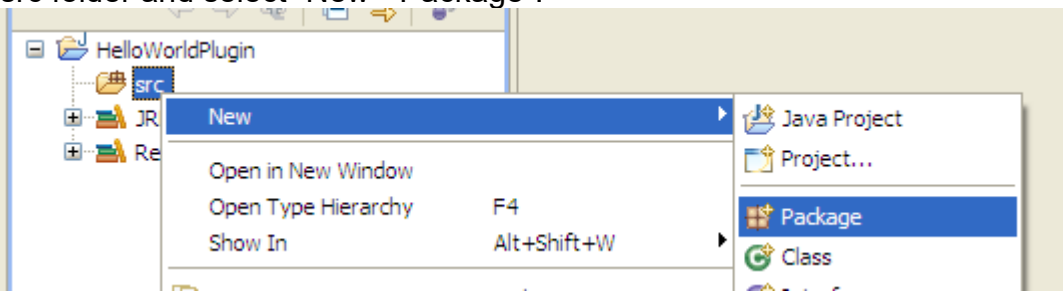


6. Click on the “Libraries” tab and then on the button “Add External JARs…”

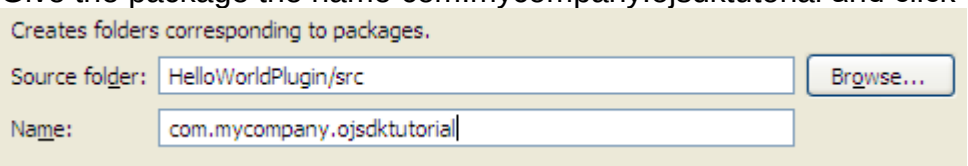


7. Browse to the com.scancoin.sdk.jar file (in C:\workspaces\oj_sdk_tutorial), select it and then click Finish to create the project.

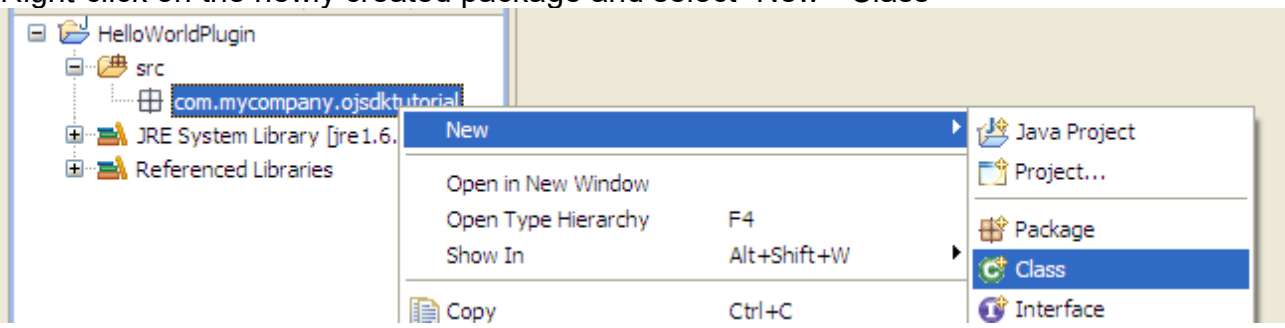
8. In the package explorer, open the HelloWorldPlugin project and right-click on the src folder and select “New->Package”:



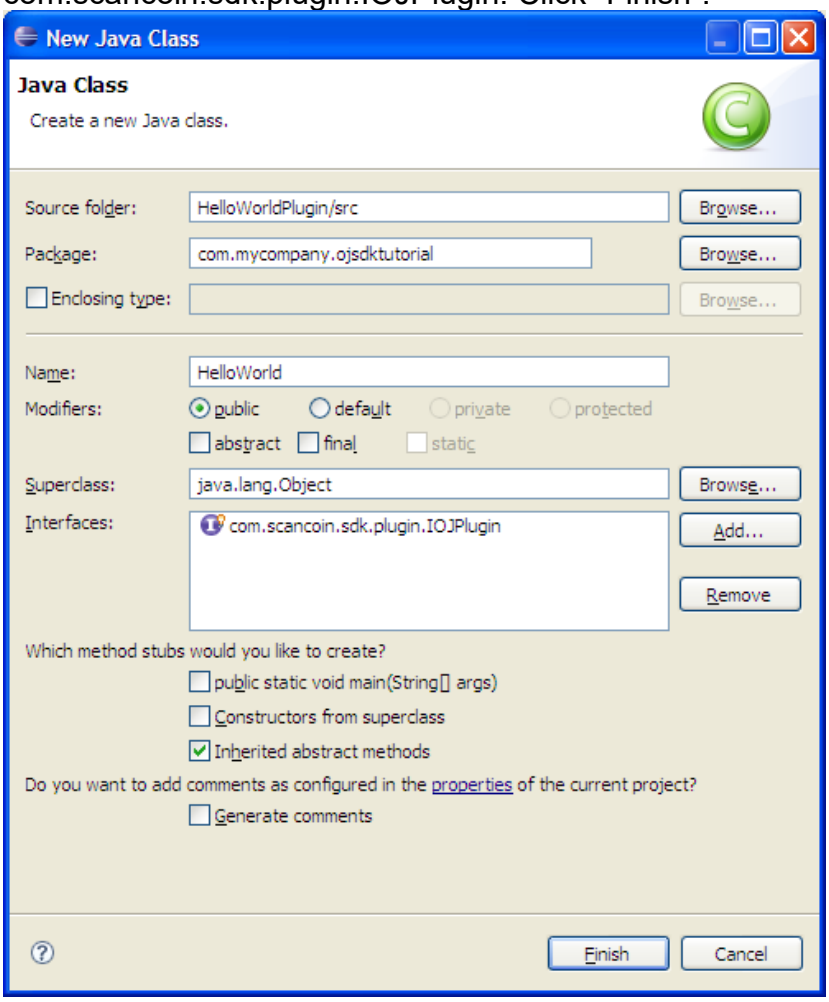
9. Give the package the name com.mycompany.ojsdktutorial and click “Finish”:



10. Right-click on the newly created package and select “New->Class”



11. The name of the class should be HelloWorld and it should implement the interface com.scancoin.sdk.plugin.IOJPlugin. Click “Finish”:



Implementing the plug-in

12. Now we shall implement the `HelloWorld` class. Follow these steps, and every time that Eclipse gives a compile error (marks something with a red underline), click on the code that gave the error to put the cursor there. Press Ctrl+1 and select "Import…" in the popup-menu.
13. Implement a private internal class called `HelloWorldFlow` which implements the interface `IOJFlow` and its method `run()`:

```
private class HelloWorldFlow implements IOJFlow {
    public void run(IOJFlowSession session) {
    }
}
```

14. Register the `HelloWorldFlow` as a boot flow in the initialize method of `HelloWorld`:

```
public void initialize(IOJRegistrator registrator) {
    try {
        registrator.registerBootFlow(new HelloWorldFlow());
    } catch(OJTooManyFlowsException e) {
    }
}
```

15. Implement the `run()` method in `HelloWorldFlow`. This will be called from OrchardJ at start-up and what we want to do is to show a screen saying "Hello World" with a Continue button and we want to wait for the user to press that button before proceeding to the normal login page. First we need to define a map (`HashMap`) with the key and key text we want to show, then we need to show the screen and after that we will wait for the user to press the key:

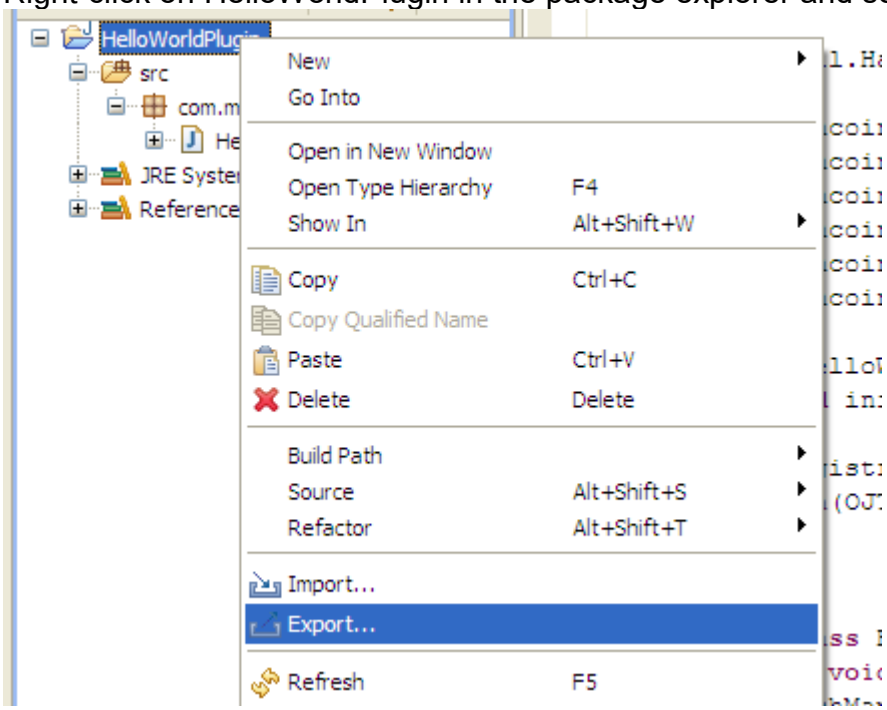
```
public void run(IOJFlowSession session) {
    HashMap<IOJUIManager.OJKey, String> key =
        new HashMap<IOJUIManager.OJKey, String>();
    key.put(IOJUIManager.OJKey.KEY_OK, "Continue");
    session.getUIManager().showTextScreen("", "Hello World",
            null, key);
    session.getUIManager().waitForKey(false, 0);
}
```

16. The entire file should look like this now:

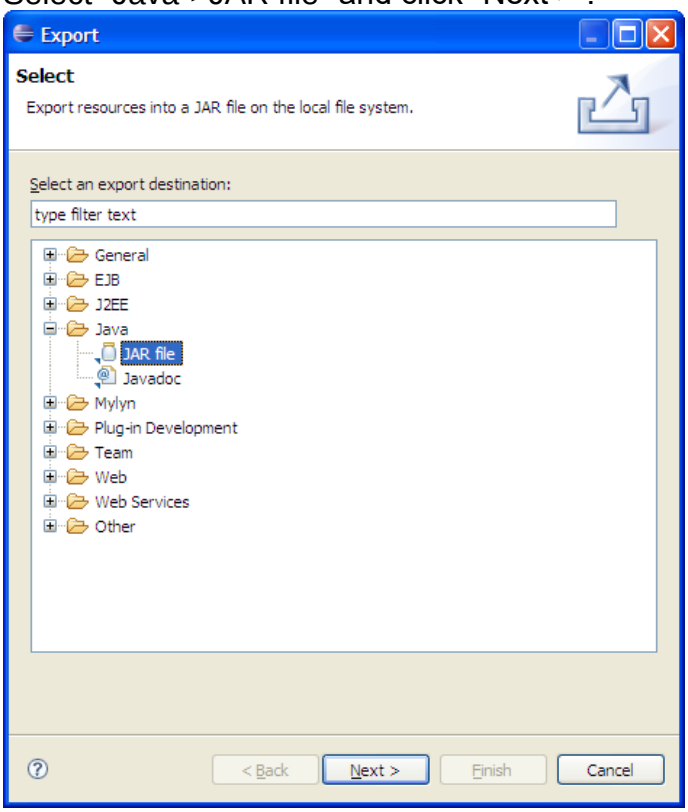
```
package com.mycompany.ojsdktutorial;

import java.util.HashMap;

import com.scancoin.sdk.oj.IOJFlowSession;
import com.scancoin.sdk.oj.IOJRegistrator;
import com.scancoin.sdk.oj.IOJUIManager;
import com.scancoin.sdk.oj.OJTooManyFlowsException;
import com.scancoin.sdk.plugin.IOJFlow;
import com.scancoin.sdk.plugin.IOJPlugin;

public class HelloWorld implements IOJPlugin {
    public void initialize(IOJRegistrator registrator) {
        try {
            registrator.registerBootFlow(new HelloWorldFlow());
        } catch(OJTooManyFlowsException e) {
        }
    }

    private class HelloWorldFlow implements IOJFlow {
        public void run(IOJFlowSession session) {
            HashMap<IOJUIManager.OJKey, String> key =
                new HashMap<IOJUIManager.OJKey, String>();
            key.put(IOJUIManager.OJKey.KEY_OK, "Continue");
            session.getUIManager().showTextScreen("", "Hello World",
                    null, key);
            session.getUIManager().waitForKey(false, 0);
        }
    }
}
```

Compiling the plug-in

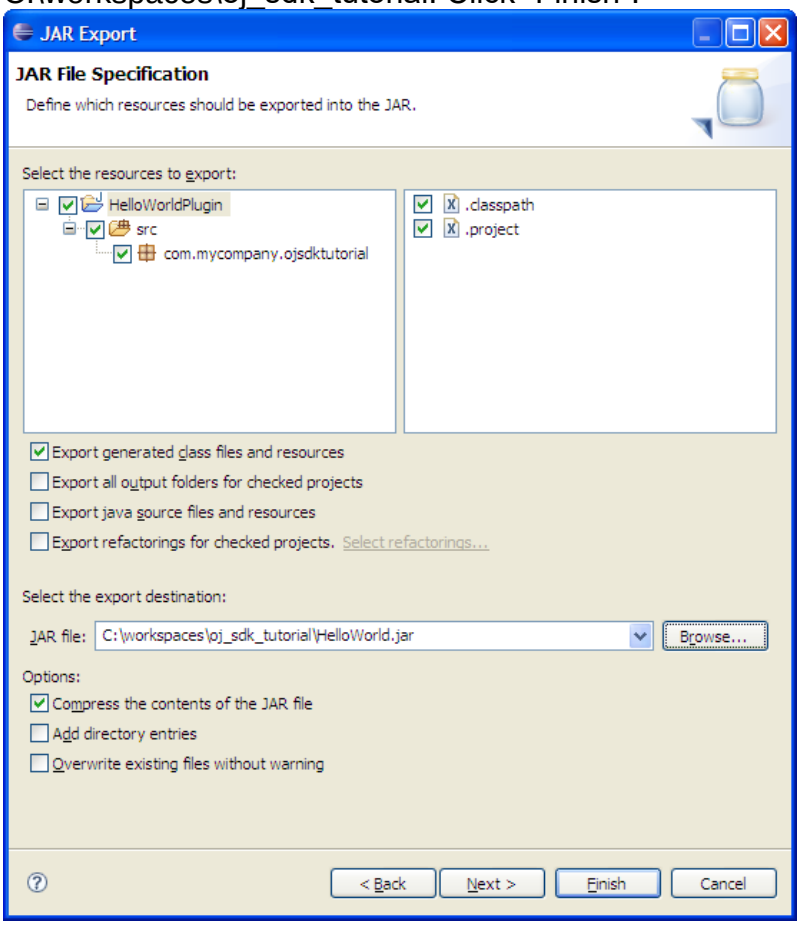
17. Right-click on HelloWorldPlugin in the package explorer and select “Export”:



18. Select “Java->JAR file” and click “Next >”:



19. Give the JAR file the name “HelloWorld.jar” and save it in C:\workspaces\oj_sdk_tutorial. Click “Finish”:



Installing the plug-in

20. Follow the steps in “Installing the plug-in” in chapter “Getting started with JDK” (page 9) to install the plug-in on an RCS.

Inside the SDK

Environment requirements

To compile a plug-in for OrchardJ you need:

- Java SE 6 (JDK 1.6) – Java SE 8 (JDK 1.8)
- The environment Path variable needs to point at the JDK bin folder

To run OrchardJ in demo mode to test plug-ins without an RCS you will need:

- An OrchardJ license (purchased from SUZOHAPP)
- The OrchardJ installer and an add-on package to run in demo mode (obtained from SUZOHAPP)
- Java JRE version 1.6 or newer
- Internet Explorer 7 or newer installed as default browser in Windows (you may have other browsers installed, but IE must be set as default browser).

How to compile plug-ins

To create a plug-in, you first need to compile it with the Java compiler (javac). If you have the source code in the src folder, the SDK lib (com.scancoin.sdk.jar) in the lib folder and you want to compile to the bin folder you write:

```
javac -sourcepath src -d bin -classpath lib\com.scancoin.sdk.jar
PathToThePluginSourceCode\ThePluginClass.java
```

And then you need to package it into a jar file. You do this with the jar program (included in the JDK). Step in to the bin folder (if you compiled as above) and write:

```
jar cf MyPlugin.jar *
```

How to install plug-ins

You can place the plug-in jar anywhere you want on the computer running OrchardJ, but a recommendation is to put it in the OrchardJ program folder under data, because if you do that then the plugins will be automatically upgraded when you upgrade OrchardJ. For example, create a folder called:
`C:\OrchardJ\data\plugins`
(if we assume that C:\OrchardJ is the OrchardJ program folder) and save the plug-in jar file there.

Now we need to tell OrchardJ to load and initialize this plug-in. This is done in the configuration file called:
`resources\configurable\configuration\com.scancoin.sdk.engine`

Open that file in a text editor (e.g. Notepad). There is one property in the file called:
`plugins`

This property lists all plug-ins to load separated by comma:
plugins = PLUGIN1,PLUGIN2,…, PLUGINn

Each plug-in is configured with the path to the jar (either absolute path or relative to the OrchardJ program folder) followed by the name of the main class (separated by semicolon). The main class should be fully qualified with package name:
`PATHTOJAR;CLASSNAME`

So in our example the file should say:
`plugins = data\plugins\MyPlugin.jar;ThePluginClass`

And if you have two plug-ins you want to load you can have it like this:
`plugins = data\plugins\MyPlugin.jar;ThePluginClass,data\plugins\MyPlugin2.jar;ThePluginClass2`

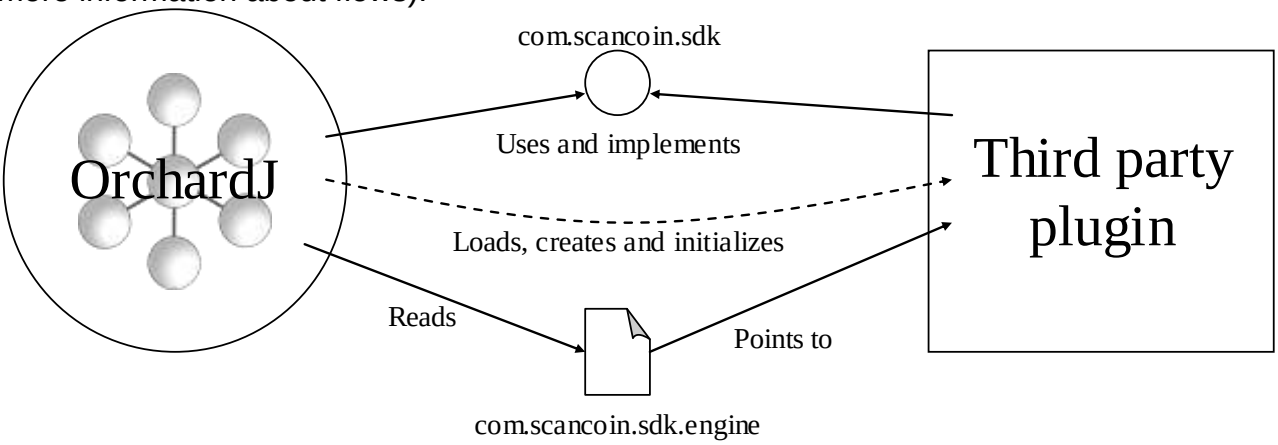
Notice that you need to restart the OrchardJ software after making these configurations.

Architecture

The OrchardJ SDK only consists of one lib called com.scancoin.sdk.jar containing interfaces and a few classes. The lib does not have any dependencies to anything except the Java Foundation Classes included in the JDK, so you do not need anything but the JDK and the SDK lib to implement a plug-in.

A plug-in must have at least one class that implements the interface `com.scancoin.sdk.plugin.IOJPlugin` (included in the SDK lib). This class can be seen as the plug-in correspondence to the standard Java main-method.

OrchardJ has a configuration file that points out all plug-in packages and all plug-in classes that should be loaded (see previous chapter for more information about this configuration file). When OrchardJ starts it will try to load all these plug-in jar files and create one instance of the configured `IOJPlugin` classes. It will call the `initialize()` method on all these instances to allow them to register their flows (see next chapter for more information about flows).



The SDK lib contains three packages:

com.scancoin.sdk.oj	Interfaces representing entities (e.g. devices, printers, receipts, users, sessions etc.) in OrchardJ. These are the interfaces that the plug-in communicates with.
com.scancoin.sdk.plugin	This package contains the interfaces that the classes in the third party plug-in implements so that OrchardJ can communicate with it.
com.scancoin.sdk.utils	Contains utility classes that can be used by the plug-in to perform common tasks.

Flows

A “flow” is what the SDK refers to when OrchardJ calls a method in the plug-in and lets that method execute and perform its tasks before it continues (there is one exception to this and that is the monitoring flow which runs in parallel to OrchardJ).

When OrchardJ starts it will call the `initialize()` method in the configured `IOJPlugin` (as explained above). To this method it passes an instance of a class that implements the interface `com.scancoin.sdk.oj.IOJRegistrator`. This object can be used by the plug-in to register its flows and to tell OrchardJ when it wants the flows to be called.

The plug-in must implement a class for each flow and let that class implement `IOJFlow` (except service flows that implements `IOJServiceFlow`). The `IOJFlow` interface has one method called:

```
void run(IOJFlowSession session)
```

That method is called from OrchardJ when the flow is entered. The flow exits as soon as the `run()` method returns (unless the flow currently has a listener that executes a call-back, in that case that listener gets to complete its task before the flow exits).

There are four different types of flows that can be registered:

Boot Flow

This flow could also be called “Idle Flow”; it just depends on how it is used.

The boot flow is called at start-up, but also when a user has logged out. It can be used to implement a new login flow or if you want to take full control of the system (e.g. to make a remote controller).

The path after the boot flow exits depends on if the boot flow logged on a user or not. If there is no user logged in it will go to the regular OrchardJ idle/login flow. If a user was logged in it will go on to select service (or the plug-in logged-in flow if there is such registered).

There can only be one boot flow registered. If the plug-in registers more than one boot flow, or if there are multiple plug-ins and more than one registers a boot flow, then you will get an exception in the registration on all but the first.

Logged-in flow

This flow is called if a user is logged in by the regular idle/login flow, or from the plug-in boot flow, just before you get to the select service flow.

You can use the logged-in flow if you want to make some additional verification of the current user, or if you want to override the regular select service flow and instead make your own, or if you only want to have one service.

When the logged-in flow exits it will either go to select service, or if the user was logged off by the logged-in flow it will go back to idle (either the normal idle/login flow, or the plug-in boot flow if that is registered).

There can only be one logged-in flow registered. If the plug-in registers more than one logged-in flow, or if there are multiple plug-ins and more than one registers a logged-in flow, then you will get an exception in the registration on all but the first.

Service flow

A service flow is represented by a button on the select service page. The flow is called by OrchardJ when the user clicks that button. If there is only one available service for a user then that service is started automatically.

Unlike the other flows, the service flow implements `IOJServiceFlow` instead of `IOJFlow`. `IOJServiceFlow` provides additional methods that control the location, text and visibility of the button. You can also disable the button (make it grey and not selectable).

When a service flow exits it will either stay in select service or go back to idle (either the normal idle/login flow, or the plug-in boot flow if that is registered) depending on whether or not the current user has the permission STAY.

Monitor flow

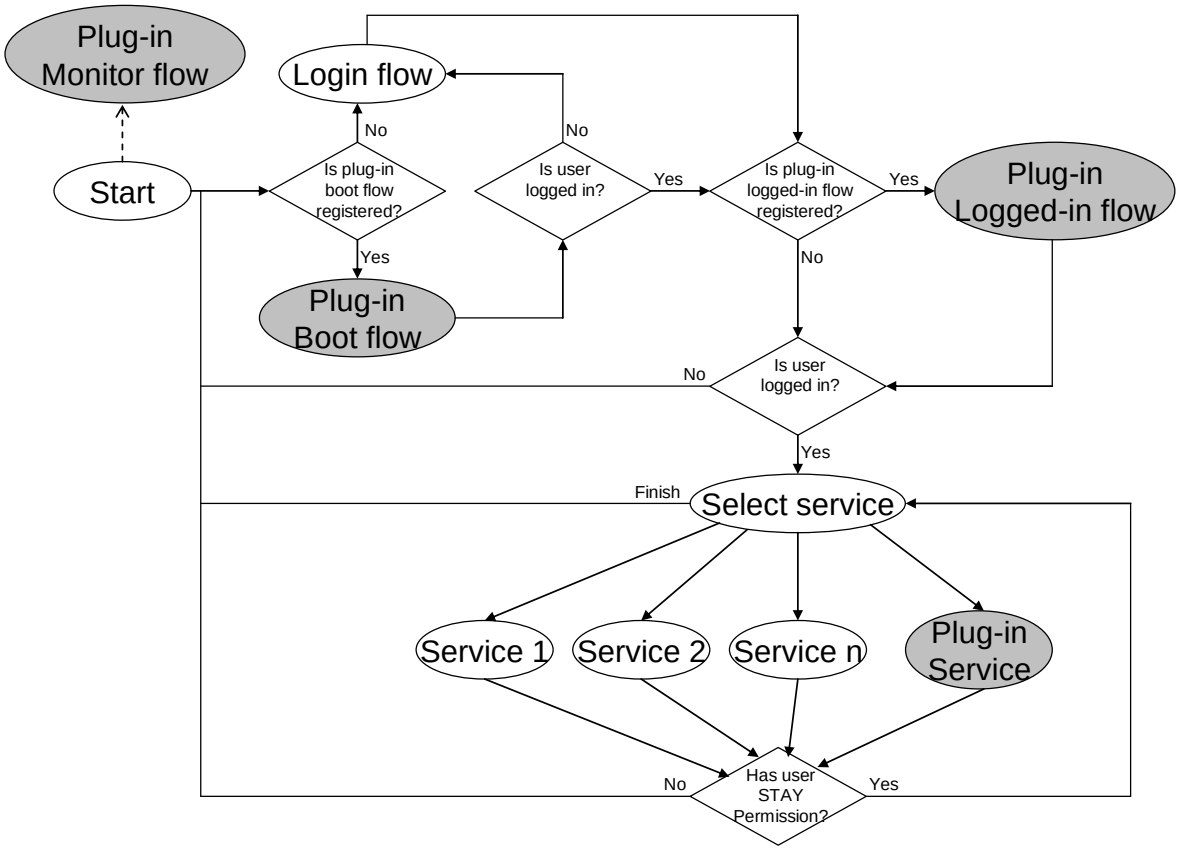
A monitor flow differs from the other flows in that OrchardJ does not wait for the flow to exit. The monitor flow is started as soon as all devices are started and it can run in parallel with all other flows. If the monitor flow exits (the `run()` method returns or throws an exception) it will never be started again until OrchardJ is restarted.

A monitor flow can typically be used to implement a remote protocol and to communicate with a remote monitoring tool.

A monitor flow has limited access to the interfaces and methods in the plug-in. In general, (with a few exceptions) it cannot tell OrchardJ to do anything, it can only listen to what happens. Please see the javadoc API for each method to see if it is accessible from the monitor flow (if you call a method that is not accessible you will get an exception).

Flow diagram

Here is a diagram that shows the OrchardJ standard flows and the plug-in flows. White flows are standard OrchardJ flows, while grey flows are plug-in flows:



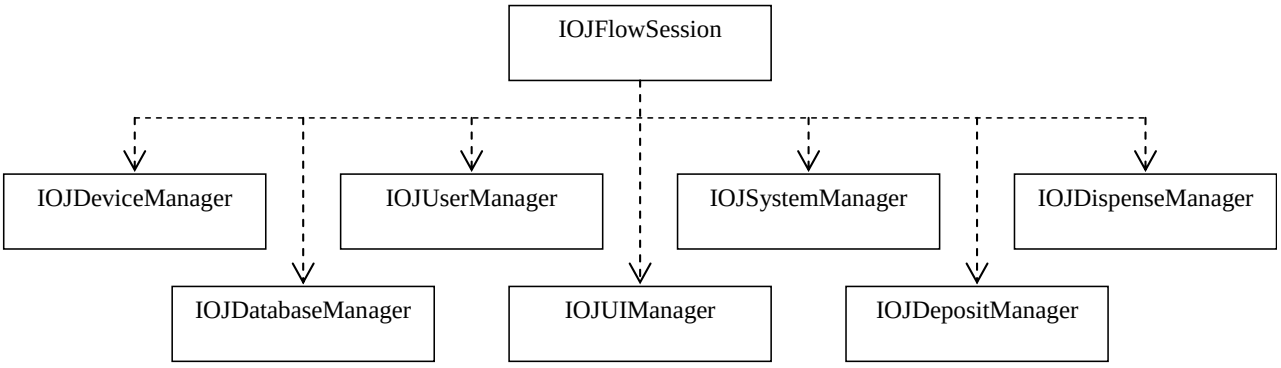
Managers

When the `run()` method of the flow is called it will receive a reference to an interface called `IOJFlowSession`. It is through this interface that the plug-in will receive access to the inner parts of OrchardJ. There are some things worth noticing about the flow session:

- Every time that a flow is entered you will get a new instance of the flow session
- The same is true for all objects retrieved from the flow session. Even if an object represents the same physical device there will be a new instance when the flow is called again.
- No method in the flow session, or in any object retrieved from the flow session, may be called after the flow exits (this could happen if you create a new thread in your flow). If you do call a method after the flow exits it will immediately throw an exception.

Through the flow session you can retrieve manager objects that are responsible for different aspects of OrchardJ. It is through these managers, or the objects retrieved from the managers that you can interact with OrchardJ.

There are currently seven different managers in the SDK:



Please see the javadoc API for more information about these managers, what you can do with them and what objects you can retrieve from them.

OrchardJ sessions

All main flows inside OrchardJ (and also the SDK plug-in flows) belong to a certain system mode. Every time that you leave one flow and enter a new flow that has a different system mode than the last a new session will be created by OrchardJ.

You can retrieve the current system mode through `IOJSystemManager.getSystemStatus()` or by registering a `IOJSystemStatusListener` on the `IOJSystemManager`.

It is important to be aware of the concept of sessions when you print the standard receipts through `IOJPrinter`. The standard deposit receipt and the standard dispense receipt both lists all transactions done in the current session. This means that if you just stay in the same flow (e.g. the boot flow) and do a lot of deposit transactions and print a deposit receipt after each transaction, you will see all past transactions on the receipt as well (which is probably not what you want). The solution is to always change flow first. For example, if you are in the boot flow and want to start a deposit you should log on a user and then exit which will take you to the logged-in state (if you have one registered). Then you can do your deposit in the logged-in state; that way it will be done in a new session.

The system modes for the plug-in flows are constant. See this table:

Plug-in flow	System mode
Boot flow	IDLE
Logged-in flow	USER
Service flow	USER

The OrchardJ flows that come before or after the plug-in flows always have a different system mode, so when a plug-in flow is called it will always have a new session. For example, between the logged-in flow and a service flow there will always be the OrchardJ select service flow which has system mode SELECT_SERVICE. That way you can be sure that the service flow will have a different session than the logged-in flow.

None of this applies to monitor flows.

Permissions

User access to different functions in OrchardJ is controlled with roles and permissions. Each user is connected to a role and each role has a set of permissions that defines what the user is allowed to do and how the system should behave for that user.

From a programming point of view, a permission is nothing more than a string. There are some fixed permissions that OrchardJ knows about and that are available in the OrchardJ configuration tool, but if you edit the file roles.xml manually you can add your own permissions if you like.

In the SDK you can check permissions on the current user by calling `hasPermission()` on the `IOJUser` interface (and you can obtain the current user by calling `getCurrentUser()` on the `IOJUserManager`).

Permissions can be used for different things, but the most common use for permissions is to decide what services that should be available in the select service screen. When you register new SDK plug-in services you have to implement the interface `IOJServiceFlow`. One of the methods in that interface is `isVisisble()` and that would be the typical place to check the permissions for a plug-in service. Here is an example on how to implement `isVisible()` on a service that checks a new permission that we call “SDK_PLUGIN”:

```
public boolean isVisisble(IOJFlowSession session) {
    IOJUser user = session.getUserManager().getCurrentUser();
    if(user == null) {
        return false;
    }
    return user.hasPermission("SDK_PLUGIN");
}
```

The operator service menu

OrchardJ usually has some form of service menu where the operator can log in to perform service tasks such as resetting devices and resolving errors. The appearance of the service menu differs between different OrchardJ applications, but in the RCS it is a separate window with the title “Status”. You use the keyboard and Alt+TAB to get to that window and then you can login with any user that has the permission “SERVICE_MODE”. However, it is only possible to login when the system is in the standard idle/login state (waiting for a user to identify herself).

But if you have implemented your own login procedure as a boot flow, and you never return to the standard idle flow, then it is important to make it possible to login to the service menu, or else the operator cannot clear errors.

When someone logs in to the service menu it is important to close all hardware devices and an out-of-use screen should be shown. It is also not possible to login to the service menu when another user is logged in. This is why it is normally only allowed when in idle.

For an SDK plug-in this is solved by registering an operator login handler by calling `setOperatorLoginHandler(IOJOperatorLoginHandler)` in `IOJUserManager`. If you do not do this, then it will not be possible to login to the service menu while inside your flow. If you have registered such a handler, then OrchardJ will call that as soon as someone tries to login to the service menu. You should decide if that should be allowed at this time or not. If you want to allow it you should return true, and then you also must make sure that you close all devices and return from your flow as soon as possible. The service menu will not appear until your flow has exited (which ensures a bad user experience if you take too long to exit). Your flow will be called again as soon as the operator logs out, but with a new instance of the flow session (and there will be a new OrchardJ session created).

Integrating external devices

There is no support in the SDK for implementing a custom device that will be handled by the regular OrchardJ flows just as those devices handled by OrchardJ. However, there is support for using custom devices in the plug-in flows.

For example, you can implement a custom identification device (card reader, RFID tag reader, finger print reader etc.) and use it in your own boot flow. See the example `ExternalLoginDevice` for more information about that.

If you want to integrate a custom deposit device or a custom dispense device, for example a cheque reader or a note recycler that is not supported by OrchardJ, then you can easily implement it in your own Deposit or Dispense flow.

When you end the deposit or dispense transaction you can add the result from the custom device and that will be saved together with the transaction and will also be included if you print the standard deposit or dispense receipts.

There is no example of this since how it looks depends greatly on the behaviour of the custom device. It also depends on if you want to do it in parallel with the other deposit devices or if it is ok to do it in sequence (the complexity increases dramatically if you want to do it in parallel).

But you can see the examples `RemoteController`, `ParallelDeposit` and `DispenseValue` to see examples of regular deposit and dispense flows. The deposit examples ends with a call to `endTransaction()` in `IOJDepositManager` and it is called with null as both parameters. If you have used a custom deposit device you simply create an `OJMoneyBag` for the deposit result from that device and pass as the first parameter to `endTransaction()`. You can also pass a second parameter which is a string value that can be used for any custom information that you like, for example a card number. The same information (an additional money bag and a custom string) can also be passed to `doDispense()` in `IOJDispenseManager` (third and fourth parameters), the only difference is that the counts in the money bag should be negative in the case of dispense.

Troubleshooting

This chapter shows some common problems you may encounter and suggested solutions to those problems.

Plug-in does not seem to start

There may be several causes for this problem.

If your plug-in is a service (registered as a service flow) and your button does not appear on the select service screen it may be that the select service screen is full. See next problem (There is no button for plug-in service) for a solution to this.

If your plug-in is registered as a boot flow or a logged-in flow it might be that there are several plug-ins activated and one of the others also register one of those flows. Either deactivate the other plug-in (in `com.scancoin.sdk.engine`) or merge the plug-ins so that you only have one boot/logged-in flow that performs the tasks of both plug-ins.

It might be that there is a configuration error in `com.scancoin.sdk.engine`. Open this file and look for an ERROR that says “com.scancoin.sdk.engine.internal.SDKService - SDK:” and that is followed by an exception:
`C:\OrchardJ\trace\orchardj_error.log`

There is no button for plug-in service

This might be because there is too many services active for the current user (role). When there are many services registered it will try to arrange them in submenus, but if there are too many even with the submenus then they will not be accepted. If the select service screen seems to be full this might be the cause. Create a different role and user with less permissions and add the plug-in service permission to that role instead.

OrchardJ does not seem to start

If you start OrchardJ and it only shows a screen that says “RCS 800” and you never get to the login screen (or the screens you show in the boot flow) it probably means that you do not have Internet Explorer 7 or 8 as default browser.
If you have another browser set as default browser you have to start Internet Explorer, select Tools->Internet Options, and go to the tab called Programs and under “Default web browser” you can click on the button “Make default”.

IOJDepositManager.beginTransaction() seems to hang

If your deposit flow seems to hang and you conclude that it is blocked in `beginTransaction()` it is probably because there is a till inside the till drawer and the system is waiting for the user to remove the till and close the drawer. You should show a screen asking the user to remove the till before calling `beginTransaction()`. If you are running in demo mode you remove the till by checking the box next to SIU_VANDALSHIELD in the window called “TIO Simulation”.

The SDK Examples

How to compile and install examples

To compile the examples, you must have JDK 1.6 installed and the path environment variable set to point at the JDK bin folder.

Open the command prompt and go to the sdk folder (e.g. C:\OrchardJ\tools\sdk) and write:

```
mkdir classes

javac -sourcepath examples -d classes -classpath
bin\com.scancoin.sdk.jar examples\com\scancoin\sdk\examples\*.java

cd classes

jar cf SDKExamples.jar com
```

Now you will have a compiled and packaged jar containing all example plug-ins in classes\SDKExamples.jar.

To install it on an RCS you need to copy SDKExamples.jar to the RCS computer and the folder:
`C:\OrchardJ\data\plugins\`
If the plugins folder does not exist, you should create it. Notice that if you installed OrchardJ to a different folder than C:\OrchardJ you should use your installed program folder instead.

Now open the SDK configuration file in a text editor (e.g. Notepad). If OrchardJ is installed to the standard folder you will find the configuration file here:
`C:\OrchardJ\resources\configurable\configuration\com.scancoin.sdk.engine`

Change the plugins property so that it looks like this (and make sure that there is not a # in front of it):
`plugins = data/plugins/SDKExamples.jar;com.scancoin.sdk.examples.XXXX`

XXXX should be replaced by the name of the example you want to activate. You can have several examples activated at the same time by comma-separating the configurations, for example like this (notice, however, that not all examples will be possible to use at the same time):

```
plugins =
data/plugins/SDKExamples.jar;com.scancoin.sdk.examples.DispenseValue,
data/plugins/SDKExamples.jar;com.scancoin.sdk.examples.ParallelDeposit
```

Some of the examples also require you to have a permission on the role that you use. Open this file:
`C:\OrchardJ\resources\configurable\configuration\roles.xml`

Locate the role that you use (select a role that doesn't have too many permissions already, because OrchardJ does not allow too many services for the same user). Add the permission to the role like this (you can see the permissions in the next chapter):
`<permission>PERMISSION_NAME</permission>`

Example plug-ins

ReceiptPrinter

This is not a complete plug-in example itself; it is a helper class that is used by some of the other examples (`BagDropper`, `ParallelDeposit` and `DispenseValue`). It will print a standard receipt, but it will also update the user interface and tell the user that a receipt is being printed and that asks the user to take the receipt. It also has some error handling and if it fails to print a receipt it shows an error screen with an image of what the receipt would have looked like.

DoorHandler

Required permission: DOOR_HANDLER

This is a simple plug-in that will show a button on the select service screen called “Door Handler”. If you press this you will see a table with all micro switches that the machine can look at, both doors and others like the till box, and the current status (opened closed) for those switches.

There are buttons for navigating up and down in this list of doors, and if the door currently selected can be opened or closed by the software there will be buttons for opening and/or closing the doors.

This example shows the new door handling in `IOJDeviceManager`. The example also shows how to show a table on the screen by implementing an `IOJTableModel`.

This example is new in 1.4.0 and replaces the RCSDoorStatus which used the old, now deprecated, I/O handling.

BagDropper

Required permission: BAG_DROPPER

This plug-in implements a simple bag drop flow. You get to scan the barcode on bags, enter the amount in the bag and then drop them into the drop safe, and you get to repeat this until you press Done and at that time a deposit receipt is printed.

This example shows you how to communicate with `IOJDropSafe` and `IOJBarcodeReader`. It also shows you how to prompt the user to input a number.

ExternalUserLogin

This example registers a boot flow that implements a new login procedure. It uses the standard OrchardJ card reader and also shows a “Start” button for manual login (if you have set the constant ENABLE_MANUAL_LOGIN to true). But it doesn’t use the OrchardJ user database (users.xml and roles.xml), instead it shows how to connect to another custom user database (in the example it is demonstrated with two hardcoded users).

This example shows how to communicate with `IOJCardReader`, how to login users with `IOJUserManager` and how to prompt for user id and passwords (pin codes). It also shows you how to handle the operator service menu and how to handle that the system is out of use by checking for errors and by registering an error listener (when you implement a completely new idle flow you must handle both the operator service menu and the out of use state).

ExternalLoginDevice

This example registers a boot flow that implements a new login procedure. This example is similar to `ExternalUserLogin`, but it uses an external identification device instead of the built in card reader, and it uses the internal OrchardJ user database instead of an external.

The external identification device could be an RFID tag reader or a finger print scanner, or anything else that can be used for user identification. But in the example it is demonstrated with a Java Swing window that has a field for input of user id and a button to simulate that the id number was read by the device.

DispenseValue

Required permission: DISPENSE_VALUE

This example shows a dispense flow where the user doesn't select an exact mix, instead she gets to enter a value (and a currency) that she wants to dispense and the plug-in tries to create a mix to match that value, and then that mix will be dispensed and a receipt printed.

The algorithm used to create that mix used to be simplified in earlier versions of the SDK, but with SDK 2.0.0 it has been updated to use the built-in mix algorithm.

RemoteController

This example shows how to implement a remote control protocol which disables the user interface on the RCS and uses it as a remote controlled coin recycler (any connected note devices will not be used in this example). This example shows a very simple remote protocol that only has commands for deposit and that is lacking a lot of error handling for the communication.

The example registers two flows: one boot flow and one logged-in flow. The boot flow waits for commands from the client and when a command is received it logs on a user and returns which will take us to the logged-in flow that performs the task (the deposit). That way there will be a new session created for each deposit command.

The example also has a server class that runs as a separate thread and handles the actual protocol.

This example also shows you how to implement a simple deposit flow that only accepts coins (only runs on one device). You will see that this is much simpler than the next example where we do parallel deposit with several devices.

To test the remote controller, you can use a freeware program called netcat. Use it to connect to port 8888 on the RCS, like this (if you run it from the RCS itself):

```
nc localhost 8888
```

Now you can enter commands in netcat as text. Press ENTER to send the commands. The responses from the RCS will be shown in netcat as separate lines.

To start a deposit, you send this command:

```
DEPOSIT:userid:username
```

The user id does not have to match one in users.xml, a new temporary user will be created with that id and name.

The RCS will send you status about the deposit and intermediate updates as it counts coins. When you are done with the deposit you send the command:

```
STOP
```

The RCS will then send you the result of the deposit and the complete amount of deposited coins. Then it will return to idle.

See the example itself for more information about the protocol.

Just like the `ExternalUserLogin` and `ExternalLoginDevice` examples this example also enables the operator service menu while it is waiting for commands. This wouldn't be necessary if you had a complete protocol where all service tasks could be done remotely.

ParallelDeposit

Required permission: PARALLEL_DEPOSIT

This example shows a more advanced deposit flow that has parallel deposit of the different deposit devices. The user interface resembles (though not exactly) the standard RCS deposit flow.

If you compare this deposit flow to that in the `RemoteController` example you will see how very much more complicated it gets when you want to do parallel deposit. The error handling gets much more complicated and you also have to be very careful with concurrency aspects and thread synchronization.

The example shows you how to use the `IOJDepositManager` and how to start deposits with both `IOJBatchDepositer` and `IOJContinuousDepositer` (and how to do it all in parallel). It shows you how to use an `IOJTableModel` to show the amount deposited and it shows you how to handle different types of deposit errors.

The `run()` method of the flow is fairly short and only shows you the outline of the flow. It calls a number of helper methods that performs the different tasks required to implement such a flow.

CustomInfoDeposit

This is an example that demonstrates how to add some pre- or post flow before or after one of the standard flows by simply hi-jacking that flow and implementing our own flow that does the additional pre-step and then launching the known flow.

What we do in our customized pre-deposit step in this example plugin is to read the current temperature, in Celsius, at the SUZOHAPP headquarters outside Chicago and add that as a custom data to the deposit transaction.

Exchange

Required permission: EXCHANGE_CASH

This is an example that demonstrate how to use the possibility to lauch known flows to create a new user flow. In this case, we create an exchange function by running a standard deposit and then launching the dispense requested amount flow.

TableEditor

*Required permission: TABLE:**

This is a small example that demonstrates how to use the new UI framework in SDK 2.0.0 (see **New UI framework**). It will also show some new permission functions and how to use OrchardJ built in configurations. It will show a screen with a table and a numpad. The title is fetched from configurations, and the rows are based on permissions. With the numpad you can enter data in the table, and finally you can print the content of the table.

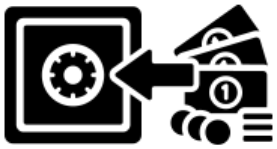
Icons for services

With the release of OrchardJ 3.0.0 (which contains SDK version 2.0.0) the services on the select service screens all have different icons that represents the function.

By default, the SDK flows will only have a star as an icon, but you can create a specialized icon for your service. Create a png image with the name of the flow, but with all spaces replaced by underscore (what is returned from IOJServiceFlow.getName().replace(' ', '_')) + ".png") and store it in this folder:
`resources/static/images/display/default`
It might be that you have to delete this file for the change to have effect:
`tmp/templates/theme.xml`
The guidelines for the icons is that they should be 196*196 in size, have a transparent background and monochrome foreground. Only the alpha-component really matters, because all icons will be redrawn to match the current theme colors.

For example, the icon for Start Deposit looks like this when you run the software:

The actual icon is a 196*196 pixels icon that is filled with a solid RGB color (for example black; 0,0,0), but with varying alpha color (alpha is the opacity/transparency). The background has alpha 0 (completely transparent) and the main part of the icon has alpha 255 (completely opac). To make it smooth the edges have an alpha value between 1 and 254. So if you look at the icon image itself (if the RGB is black) it looks like this:



SDK 2.0.0

The SDK version 2.0.0 was released together with OrchardJ version 3.0.0. This chapter describes all the news in SDK 2.0.0 and how to use them.

New UI framework

Examples: TableEditor

The user interface possibilities in the old SDK were quite limited. You could only show static screens (except for some input screens), and they were limited to just a few layouts. If you needed a more advanced GUI you either needed to generate a complete HTML page and use the custom screen, or build a standard Java GUI (e.g. Swing) and display that frame instead of the OrchardJ GUI, which is both cumbersome and not future proof.

With SDK 2.0.0 you can build dynamic GUI for OrchardJ with layout managers in the same way as you do in Java AWT or Swing. You use the same containers and layout managers as you do when you create a Java Swing GUI, but you populate it with special components created through the SDK.

The base for all this is a class called OJScreen. You should see this class as the JFrame of this framework. It inherits from JPanel and can be used as such to populate it with components. You can instantiate the OJScreen itself and add components to it, but the preferable way is to inherit it and make your own screen class. When you have a populated screen, you call IOJUIManager.showScreen() to display it.

When you populate the screen, you can use all the standard layout manages from AWT and Swing, and you can use all the containers such as JPanel and JContainer. You can also add standard AWT/Swing components like JButton or JLabel, but they will never be shown in the OrchardJ GUI. The size of these components will however be considered when doing the layout, and sometimes it may make sense to add them just for layout purposes (you can see a usage of a JLabel in this way in the TableEditor example).

One thing that differs between the SDK UI framework and AWT/Swing is that you cannot instantiate the components directly (except for OJScreen there are no classes, only interfaces, for the components). So instead of instantiating a component class you need to use a factory interface (called IOJComponentFactory). You receive on instance of this component factory by calling IOJUIManager.getComponentFactory().

If you don’t know how to build GUI with Swing and layout managers there are many books, web pages and the Java documentation itself; it will not be described in deep detail in this manual. For an example of how it is used with the SDK, please see the TableEditor example.

Notice that the old, simple GUI functions still exist, and if all you need is a simple message screen or input screen, it is probably better to use them than to create your own screen with the new framework.

Launching known flows

Examples: CustomInfoDeposit and Exchange

With version 2.0.0 of the SDK it is possible to launch one of the standard OrchardJ flows/services from within an SDK plugin flow. This can be used to build an aggregated flow, or to add a custom action before or after a standard OrchardJ flow.

An aggregated flow, i.e. a flow that is a sequence of other standard flows, can be used for some different purposes:

- To create a completely new function, e.g. to create an Exchange as a combination of a standard deposit flow and a standard dispense flow (see the Exchange example).
- To force users to follow a certain custom process. For example, someone wants to enforce a replenishment process where CIT always prints machine total, moves notes, empties boxes and finally prints machine total again.

The other case is that you want some new action before or after a standard flow, e.g. to set some custom data before a deposit like in the CustomInfoDeposit example. In this case, you implement your own flow with the same icon and name as the standard flow and hide the standard flow (use blockPermssion() as in the CustomInfoDeposit example). From your flow, you launch the standard flow and add your custom actions before or after.

The main way of launching a standard flow is by calling executeKnownFlow() in IOJSystemManager supplying an enum to indicate what known flow to launch. This method is blocking and will block your plugin flow until the standard flow exits. With this method you can launch most flows that can be launched from select service (the enum contains more than 50 different known flows, see the API reference documentation for OJKnownFlow in IOJSystemManager). You can, however, not pass any data to the flows or receive any results, to know what happened in the flow you need to go to the IOJDatabaseManager to investigate new transactions stored.

There is one special flow that can be launched with more control, and that is a standard deposit. You can launch a deposit with the method described above, but there is also another way. IOJDepositManager has a method called runDepositFlow() which will also block while a standard deposit is done, but it takes two parameters (the transaction type to use and whether or not to print a receipt) and it returns a money bag with the content that was deposited.

Miscellaneous

Permissions

Examples: CustomInfoDeposit and TableEditor

Previously there were only one way to check if a user had a certain permission and that was with the hasPermission() method on IOJUser. However, some permissions are parameterized (e.g. “FREE_FLOAT_AMOUNT_LIMIT:1000”) and in some cases you don’t know what string to use (you can’t call with all possible values in the previous example).

For this reason there is now a getPermissions() method where you supply a prefix. In the above example, you can supply “FREE_FLOAT_AMOUNT_LIMIT:” as a prefix and you will get all enabled permissions for the user that starts with that prefix. See the TableEditor example for a usage of this.

Two other new permission methods in IOJUser are blockPermission() and unblockPermission(). This will temporarily hide a permission for a user so that it appears like she doesn’t have that permission. This can be useful for “hijacking” a flow where you

want to replace a standard flow with your own, see more in the Launching known flows chapter and the CustomInfoDeposit example.

Custom data

Examples: CustomInfoDeposit

If you wanted to provide some custom information with a transaction prior to SDK 2.0.0 you could supply a parameter called extra data (a string limited to 255 characters). This string got stored with the transaction and sent with the RM protocol.

However, OrchardJ itself doesn't use this extra data and therefore it is usually not implemented in enterprise software solutions using the RM protocol. OrchardJ itself uses something called custom data instead, which similarly can be used to supply custom strings with a transaction, but instead of being just a single string you provide a map with key-value pairs which allows much more custom information.

With SDK 2.0.0 you can now use custom data from the SDK itself as well, while the old "extra data" is still there for backwards compatibility.

Another difference is that you don't supply the custom data when you create the transaction like you do with extra data. Instead you set the custom data on a user object, and then it will be used for all transactions created on that user in this session, whether they are created by the SDK plugin or by OrchardJ itself. This means that you can set custom data on a user in a logged in flow which will be used by the native OrchardJ flows, and it can also be used when launching a standard flow from the SDK plugin (see Launching known flows).

Money mix algorithm

Examples: Exchange and DispenseValue

Sometimes you want to create an optimal mix of coins and notes to dispense a certain value. At first thought this seems like an easy algorithm, just take as many of the highest available denomination as possible without exceeding the value and then proceed with the second-highest denomination and so on (this was actually how it was implemented in the DispenseValue example prior to 2.0.0).

The problem with this algorithm is if you, for example, want to create a mix for €110 when all you have are €50 notes and €20 notes. In this case you will take 2 €50 notes, reaching €100, but then you have no possibility of fulfilling the last €10. It might even be that the system is full of 1 cent coins in which case you will dispense 2 €50 notes and 1000 coins, where you could have reached the full amount with 1 €50 note and 3 €20 notes.

With SDK 2.0.0 there is a built-in algorithm that takes these cases into account, and also the available cash in the machine and all built-in dispense limits.

Just call createMix() in IOJDispenseManager and supply a money bag that has the value you want to split. This value is taken as a money bag even though it is just a value you want to split, and the reason for this is that it is often convenient to use a money bag, and it will handle different currencies automatically. But it is only the total value (per currency) of the money bag that is used. The result of the split is given as an OJMixResult. This holds the exact mix, if an exact mix could be found, and also the best underpay and the best overpay (these are usually just used if an exact mix couldn't be found).

Use OrchardJ trace files

Examples: CustomInfoDeposit

When you develop software you often want to write trace log files from the program, both to debug while developing, but also to analyse error reports from the field. With SDK 2.0.0 it is possible for an SDK plugin to log into the native trace files of OrchardJ (i.e. /OrcahrdJ/trace/orchardj.log). OrchardJ uses a third-party library from Apache called log4j to write these trace files.

Call createLogger() in IOJSystemManager to get an object of type IOJLogger which you can use to log to the OrchardJ trace files. When you call createLogger() you supply the SDK class using the logger, this class name is written in the log rows and can be used to find the trace that is relevant for you.

Use OrchardJ configurations

Examples: TableEditor

If you require configurations for your plugin you no longer need to write your own file format and file parsing for those configurations, instead you can make use of the OrchardJ configuration files.

There is already the configuration file called com.scancoin.sdk.engine, which is where the plugin is activated. Now you can add your custom configuration in this file as well and just ask the SDK API for the setting of these configurations.

Call getConfiguration() in IOJSystemManager to get a Properties object containing all the configurations belonging to your plugin. To determine what configurations that belongs to you, you supply the main class of your plugin and the fully qualified name of that class should be used as prefix in the configuration file. E.g. if the main plugin class (which is supplied to getConfiguration()) is called com.mycom.myplugin.MyPlugin and you have a property called FlowTimeout, then it should be written like this in the com.scancoin.sdk.engine:

```
com.mycom.myplugin.MyPlugin.FlowTimeout = 1000
```

In the Properties object returned it will just be called FlowTimeout though.